

PlantCare – Prolog Expert System

Name: Samarth Kadam

PRN: 202401040131

Batch: A2

❖ Title

Implementation of PlantCare Expert System using Artificial Intelligence and Prolog

❖ Aim

To design and implement an intelligent **PlantCare Expert System** using Prolog that provides plant-care recommendations based on plant type, watering requirements, sunlight requirements, weather conditions, and plant symptoms.

❖ Objectives

1. To understand the concept of an Expert System.
2. To understand the use of **Knowledge Representation in Prolog**.
3. To create a knowledge base containing information about different plants.
4. To identify the watering and sunlight requirements of plants.
5. To classify plants according to their types.
6. To provide weather-based plant-care advice.
7. To identify common plant problems and provide possible solutions.
8. To implement a conversational PlantCare assistant using Prolog.
9. To understand the use of facts, rules, variables, and predicates in Prolog.
10. To demonstrate how an AI-based expert system can provide recommendations using a knowledge base.

❖ Outcomes

After completing this experiment, the following outcomes were achieved:

- Successfully implemented a **PlantCare Expert System using Prolog**.
- Created a knowledge base containing information about **10 different plants**.
- Stored watering requirements for different plants.
- Stored sunlight requirements for different plants.
- Classified plants into categories such as flowering, succulent, indoor, and herb.
- Implemented weather-based plant-care recommendations.
- Implemented plant-problem diagnosis and basic solutions.
- Implemented conversational queries for interacting with the PlantCare assistant.
- Implemented multiple rules for retrieving plant information.
- Understood how Prolog uses facts and rules to perform logical reasoning.
- Implemented a fallback response when the system does not understand the user's query.

❖ Conceptual Details

- Prolog Facts
- Facts represent information that is already known by the system.
- Example:
- `plant(rose).`
- This means that rose is a plant known to the system.
- Prolog Predicates
- Predicates represent relationships between entities.
- For example:
- `water_need(cactus, low).`

- represents the relationship between cactus and its low water requirement.
- Prolog Rules
- Rules determine what the system should do when a particular condition is satisfied.
- For example:
- `reply([how,much,water,does,Plant,need | _]) :-`
 - `water_need(Plant, Need),`
 - `write(Plant),`
 - `write(' needs '),`
 - `write(Need),`
 - `write(' watering.'),`
 - `nl,`
 - `plantcare_loop.`
- When the user asks about the water requirement of a plant, Prolog searches the knowledge base and returns the corresponding information.

❖ **Algorithm**

- Algorithm: PlantCare Expert System
- Step 1: Start the program.
- Step 2: Create the PlantCare knowledge base.
- Step 3: Store the names of known plants.
- Step 4: Store watering requirements for each plant.
- Step 5: Store sunlight requirements for each plant.
- Step 6: Store plant categories such as flowering, succulent, indoor, and herb.
- Step 7: Store weather-based plant-care recommendations.
- Step 8: Store common plant problems and their possible solutions.

- Step 9: Start the PlantCare conversational loop.
- Step 10: Accept a query from the user.
- Step 11: Match the query with an appropriate Prolog rule.
- Step 12: If the query asks about a plant, search the plant knowledge base.
- Step 13: If the query asks about watering, retrieve the watering requirement.
- Step 14: If the query asks about sunlight, retrieve the light requirement.
- Step 15: If the query asks about plant type, retrieve its category.
- Step 16: If the query asks about weather, provide weather-based advice.
- Step 17: If the user describes a plant problem, provide the corresponding solution.
- Step 18: If the query is not recognized, provide a random clarification response.
- Step 19: Continue the conversation until the user enters bye.
- Step 20: Stop the program.

❖ Program

```
/* knowledge base */
```

```
/* PLANT INFORMATION */
```

```
plant(rose).
plant(tulip).
plant(cactus).
plant(aloevera).
plant(moneyplant).
plant(mint).
plant(basil).
plant(sunflower).
plant(lily).
plant(fern).
```

```
/* PLANT WATERING NEEDS */
```

```
water_need(rose, medium).  
water_need(tulip, medium).  
water_need(cactus, low).  
water_need(aloevera, low).  
water_need(moneyplant, medium).  
water_need(mint, high).  
water_need(basil, high).  
water_need(sunflower, medium).  
water_need(lily, high).  
water_need(fern, high).
```

```
/* PLANT LIGHT NEEDS */
```

```
light_need(rose, high).  
light_need(tulip, high).  
light_need(cactus, high).  
light_need(aloevera, high).  
light_need(moneyplant, medium).  
light_need(mint, medium).  
light_need(basil, high).  
light_need(sunflower, high).  
light_need(lily, medium).  
light_need(fern, low).
```

```
/* PLANT TYPES */
```

```
plant_type(rose, flowering).  
plant_type(tulip, flowering).  
plant_type(cactus, succulent).  
plant_type(aloevera, succulent).  
plant_type(moneyplant, indoor).  
plant_type(mint, herb).  
plant_type(basil, herb).  
plant_type(sunflower, flowering).  
plant_type(lily, flowering).  
plant_type(fern, indoor).
```

```
/* WEATHER BASED CARE */
```

```
weather_care(hot, 'Water the plant carefully and keep it hydrated.').  
weather_care(cold, 'Avoid excessive watering during cold weather.').  
weather_care(rainy, 'Reduce watering because the soil may already be wet.').  
weather_care(normal, 'Follow the normal watering schedule.').
```

```
/* PLANT SYMPTOMS */
```

```
plant_problem(yellow_leaves, 'The plant may be getting too much water.').  
plant_problem(dry_leaves, 'The plant may need more water.').  
plant_problem(brown_leaves, 'The plant may need better watering or humidity.').  
plant_problem(no_growth, 'The plant may need more sunlight or nutrients.').  
plant_problem(wilting, 'The plant may need water and proper sunlight.').
```

```
/* DYNAMIC LIST FOR RANDOM RESPONSES */
```

```
:- dynamic confuse/1.
```

```
confuse([  
    'Sorry, I did not understand.',  
    'Please tell me more about your plant.',  
    'Hmm... Let me think about your plant.',  
    'I need more information to help you.'  
]).
```

```
/* START PLANTCARE */
```

```
plantcare :-  
    write('Hello! I am PlantCare. '), nl,  
    write('Your personal plant care assistant. '), nl, nl,  
    plantcare_loop.
```

```
/* PLANTCARE LOOP */
```

```
plantcare_loop :-  
    write('PlantCare:- '),  
    read(Input),  
    reply(Input).
```

```
/* HELLO */
```

```
reply([hello | _]) :-  
    write('Hello! Nice to meet you. '), nl,  
    write('How can I help you with your plant? '), nl, nl,  
    plantcare_loop.
```

```
reply([hi | _]) :-
```

```
write('Hi! Which plant do you want help with?'), nl, nl,  
plantcare_loop.
```

```
/* ASK WHAT PLANTCARE CAN DO */
```

```
reply([what,can,you,do | _]) :-  
    write('I can provide plant care information, '), nl,  
    write('watering advice, light requirements and problem solutions. '), nl, nl,  
    plantcare_loop.
```

```
/* CHECK PLANT */
```

```
reply([do,you,know,plant,Plant | _]) :-  
    plant(Plant),  
    write('Yes, I know about '),  
    write(Plant),  
    write('. '), nl, nl,  
    plantcare_loop.
```

```
/* CHECK WATER REQUIREMENT */
```

```
reply([how,much,water,does,Plant,need | _]) :-  
    water_need(Plant, Need),  
    write(Plant),  
    write(' needs '),  
    write(Need),  
    write(' watering. '), nl, nl,  
    plantcare_loop.
```

```
/* CHECK LIGHT REQUIREMENT */
```

```
reply([how,much,sunlight,does,Plant,need | _]) :-  
    light_need(Plant, Light),  
    write(Plant),  
    write(' needs '),  
    write(Light),  
    write(' sunlight. '), nl, nl,  
    plantcare_loop.
```

```
/* CHECK PLANT TYPE */
```

```
reply([what,type,is,Plant | _]) :-  
    plant_type(Plant, Type),
```

```
write(Plant),  
write(' is a '),  
write(Type),  
write(' plant. '), nl, nl,  
plantcare_loop.
```

```
/* WEATHER ADVICE */
```

```
reply([weather,is,Weather | _]) :-  
    weather_care(Weather, Advice),  
    write(Advice), nl, nl,  
    plantcare_loop.
```

```
/* PLANT PROBLEM */
```

```
reply([my,plant,has,Problem | _]) :-  
    plant_problem(Problem, Solution),  
    write(Solution), nl, nl,  
    plantcare_loop.
```

```
/* FIND LOW WATER PLANTS */
```

```
reply([which,plants,need,little,water | _]) :-  
    water_need(Plant, low),  
    write(Plant),  
    write(' needs little water. '), nl, nl,  
    plantcare_loop.
```

```
/* FIND HIGH WATER PLANTS */
```

```
reply([which,plants,need,more,water | _]) :-  
    water_need(Plant, high),  
    write(Plant),  
    write(' needs more water. '), nl, nl,  
    plantcare_loop.
```

```
/* FIND INDOOR PLANTS */
```

```
reply([which,plants,are,indoor | _]) :-  
    plant_type(Plant, indoor),  
    write(Plant),  
    write(' is suitable as an indoor plant. '), nl, nl,  
    plantcare_loop.
```

```

/* FIND FLOWERING PLANTS */

reply([which,plants,have,flowers | _]) :-
    plant_type(Plant, flowering),
    write(Plant),
    write(' is a flowering plant. '), nl, nl,
    plantcare_loop.

/* PLANT INFORMATION */

reply([tell,me,about,Plant | _]) :-
    plant(Plant),
    water_need(Plant, Water),
    light_need(Plant, Light),
    plant_type(Plant, Type),
    write(Plant),
    write(' is a '),
    write(Type),
    write(' plant. '), nl,
    write('Water requirement: '),
    write(Water), nl,
    write('Light requirement: '),
    write(Light), nl, nl,
    plantcare_loop.

/* BYE */

reply([bye | _]) :-
    write('It was nice helping you with your plants!'), nl,
    write('PlantCare says: Keep growing! Bye!'), nl.

/* RANDOM REPLY IF QUERY IS NOT UNDERSTOOD */

reply([_ | _]) :-
    retract(confuse([Next | Rest])),
    append(Rest, [Next], NewList),
    asserta(confuse(NewList)),
    write(Next), nl, nl,
    plantcare_loop.

```

❖ Output


plantcare.




Hello! I am PlantCare.
Your personal plant care assistant.

PlantCare:-

Hello

Hello! Nice to meet you.
How can I help you with your plant?

PlantCare:-

[what,can,you,do].

I can provide plant care information,
watering advice, light requirements and problem solutions.

PlantCare:-

[do,you,know,plant,rose].

Yes, I know about rose.

PlantCare:-

[how,much,water,does,rose,need].

rose needs medium watering.

PlantCare:-

[how,much,sunlight,does,lily,need].

lily needs medium sunlight.

PlantCare:-

[how,much,sunlight,does,lily,need].

lily needs medium sunlight.

PlantCare:-

[what,type,is,cactus].

cactus is a succulent plant.

PlantCare:-

[weather,is,cold].

Avoid excessive watering during cold weather.

PlantCare:-

[my,plant,has,yellow_leaves].

The plant may be getting too much water.

PlantCare:-

[tell,me,about,aloevera].

aloevera is a succulent plant.
Water requirement: low
Light requirement: high

❖ Result and Analysis

The PlantCare Expert System was successfully implemented using Prolog. The system uses a structured knowledge base containing information about different plants and applies logical rules to answer user queries.

The system successfully identifies known plants and provides information about their watering and sunlight requirements. It also classifies plants into different categories such as flowering, succulent, indoor, and herb.

The system provides additional recommendations based on weather conditions. For example, during rainy weather, it recommends reducing watering because the soil may already be wet.

The system can also identify common plant problems such as yellow leaves, dry leaves, brown leaves, no growth, and wilting, and provide appropriate basic advice.

The conversational interface allows the user to interact with PlantCare using simple natural-language-like queries represented as Prolog lists.

The system also contains a fallback mechanism. When the user enters an unsupported query, PlantCare provides a response asking for more information rather than terminating unexpectedly.

Overall, the experiment demonstrates how Artificial Intelligence, knowledge representation, logical reasoning, facts, predicates, and rules can be combined to develop a simple domain-specific expert system.

❖ Conclusion

The PlantCare Expert System was successfully implemented using Prolog. The system demonstrates the basic concepts of Artificial Intelligence and Expert Systems by representing plant-care knowledge through facts and rules.

The system can provide information about plant types, watering requirements, sunlight requirements, weather-based care, and common plant problems. It can also interact with the user through a simple conversational interface.

Hence, the experiment successfully demonstrates how Prolog can be used to develop a rule-based intelligent expert system for plant care.

❖ **Github Link:**

https://github.com/Samu7709/AIML_Lab_Assignments_CA